Сложение

Для сложения двух чисел предназначена команда ADD. Она работает как с числами со знаком, так и с числами без знака (это особенность дополнительного кода).

Операнды должны иметь одинаковый размер (нельзя складывать 16- и 8-битное значение).

Результат помещается на место первого операнда. В общем, эти правила справедливы для большинства команд.

После выполнения команды изменяются флаги, по которым можно определить характеристики результата:

- Флаг *CF* устанавливается, если при сложении произошёл перенос из старшего разряда. Для беззнаковых чисел это будет означать, что произошло переполнение и результат получился некорректным.
- Флаг *OF* обозначает переполнение для чисел со знаком.
- Флаг *SF* равен знаковому биту результата (естественно, для чисел со знаком, а для беззнаковых он равен старшему биту и особо смысла не имеет).
- Флаг *ZF* устанавливается, если результат равен 0.
- Флаг *PF* -- признак чётности, равен 1, если результат содержит нечётное число единиц.

Примеры:

```
    add ax,5          ;AX = AX + 5
    add dx,cx         ;DX = DX + CX
add dx,cl         ;Ошибка: разный размер операндов.
```

Вычитание

Вычитание выполняется с помощью команды SUB. Результат также помещается на место первого операнда и опять же выставляются флаги. Единственная разница в том, что происходит вычитание, а не сложение.

На самом деле вычитание в процессоре реализовано с помощью сложения. Процессор меняет знак второго операнда на противоположный, а затем складывает два числа. Если вам необходимо в программе поменять знак числа на противоположный, можно использовать команду NEG. У этой команды всего один операнд.

Примеры:

```
    sub si,dx         ;SI = SI - DX
neg ax            ;AX = -AX
```

Инкремент и декремент

Очень часто в программах используется операция прибавления или вычитания единицы. Прибавление единицы называется инкрементом, а вычитание -- декрементом. Для этих операций существуют специальные команды процессора: INC и DEC. Обратите внимание, что эти команды не изменяют значение флага *CF*.

Пример программы

Чтобы всё стало совсем понятно, напишем небольшую программу. Требуется вычислить значение формулы: *e=a-(b+c-1)+(-d)*. Все числа являются 8-битными целыми со знаком. Объявим их после кода и придумаем какие-нибудь значения.

Пример:

```
1  use16               ;Генерировать 16-битный код
2  org 100h            ;Программа начинается с адреса 100h
3      mov al,[a]      ;Загружаем значение a в AL
4      mov ah,[b]      ;Загружаем значение b в AH
5  add ah,[c]       ;AH = AH + c = b+c
6      dec ah          ;AH = AH - 1 = b+c-1
7      sub al,ah       ;AL = AL - AH = a-(b+c-1)
8      mov cl,[d]      ;CL = d
9      neg cl          ;CL = -CL = -d
10     add al,cl       ;AL = AL + CL = a-(b+c-1)+(-d)
11 mov [e],al       ;Сохраняем результат в e
12     mov ax,4C00h    ;\
13     int 21h         ;/ Завершение программы
14 ;-------------------------------------------------------
15 a db 2
16 b db 3
17 c db 5
18 d db -8
19 e db ?
20
21
```

Квадратные скобки означают, что операнд находится по адресу, указанному внутри этих скобок. Так как вместо имени переменной FASM подставляет её адрес, то такая запись позволяет прочитать или записать значение переменной.
Запустив программу в Turbo Debugger, можно посмотреть её выполнение по шагам. Значения переменных можно увидеть в окне дампа памяти. Для этого нужно кликнуть правой кнопкой в этом окне и выбрать в меню пункт *Goto...*. Переменные начинаются в памяти с адреса 011Fh (этот адрес в первой команде).
В этих байтах легко угадываются наши переменные:

Упражнение
Напишите программу для вычисления формул.
Все числа 16-битные целые со знаком.
Запустите в отладчике и проверьте правильность вычисления.
Продемонстрируйте преподавателю.

1	Задание	2	Задание
	k=m+1-(n-1-r)		*k=m+r-1+(m+1-n)*
	k=m-3+(n+1+r)		*k=m-1+n-(n-m+r)*
	k=m+3-(n-m+r)-1		*k=r-m+1-(r-n-1+r)*
	k=m+1-(m+1+n-r)		*k=n-m-1+(r+1-n)*
	k=r+m-(n-r-1)		*k=n-r+1-(m-1+r)*
	k=m+n-(n-2+r)+1		*k=r-n+1-(m+r-1)*